

Object Oriented Programming Lab



Assignment:

Semester End Project

Submitted By

Alesha Shahid 09-131242-005

Anis Ahmad 09-131242-007

Submitted to
Sir Faisal Zia

Section: BSE-2A
Department Of Software Engineering
Bahria University H-11

Contents

1. Executive Summary	2
2. Problem Statement	3
3. Proposed	3
4. Core Features	4
5. Object-Oriented Design	6
6. System Architecture	11
7. Implementation Details	15
8. Code Snippets	19
Snippet 1, Abstract Entity Base Class	19
Snippet 2, Runtime Polymorphism in the Game Loop	19
}	20
Snippet 3, Encapsulation and Constructors/Destructors	20
Snippet 4, SQLite Leaderboard with ScoreManager	21
9. Testing	25
10. Data Storage	26
11. Technology Stack	27
12. Challenges and Solutions	28
13. Scope and Limitations	30
14. Conclusion	32
15. References	33

1. Executive Summary

Sea of Chaos is a fully playable 2D arcade game developed in C++ using the **Raylib 5.0 graphics library**. *Inspired by the classic Space Invaders*, it is reimaged in an underwater setting in which the player controls a submarine defending the ocean floor against advancing waves of hostile sea creatures, Pufferfish, Clownfish, Sharks, and Anglerfish, firing torpedoes upward while avoiding enemy counter-fire.

The project is a practical demonstration of all four core Object-Oriented Programming principles. An abstract Entity base class establishes a uniform interface for every game object. **Runtime polymorphism** through C++ virtual dispatch drives the main game loop, calling Update(), Draw(), and GetBounds() uniformly across fundamentally different object types. **Encapsulation** is consistently enforced through private members and controlled public interfaces. **Inheritance** eliminates code duplication by placing shared state and behaviour in the Entity base, from which all seven entity subclasses derive.

Persistent leaderboard data is stored using SQLite3, a lightweight embedded relational database that goes beyond flat file storage to demonstrate real-world database integration. The game includes multiple waves of increasing difficulty, a power-up system with three distinct effects, destructible coral reef bunkers with block-level damage, a bonus Blue Whale enemy, and a complete user interface with menus, wave-clear screens, and a game over overlay, all built on a clean, modular OOP architecture.

2. Problem Statement

Most beginner C++ projects demonstrate OOP through dry, abstract examples such as bank accounts, student record systems, or library management systems. While these convey theory, *they fail* to show why OOP is the natural solution to real, complex problems. It is difficult to appreciate how classes, inheritance, and polymorphism cooperate in a live system when the examples are trivial and static.

Sea of Chaos was chosen specifically because a Space Invaders-style game is a problem domain where OOP is not forced, **it is the obvious answer**. Every object in the game naturally becomes its own class with its own data and behavior. The game loop naturally calls the same interface methods on fundamentally different types of objects. Polymorphism, encapsulation, and inheritance emerge from the problem itself rather than being artificially imposed onto it.

3. Proposed Solution

Sea of Chaos implements the complete Space Invaders gameplay loop inside an underwater theme. **The player** controls a submarine using the left and right arrow keys and fires torpedoes upward with the spacebar. **Five rows of sea creatures** advance in synchronized formation, reversing direction and descending when they reach the screen edge, and fire torpedoes downward at the submarine at randomized intervals.

Four destructible coral reef bunkers provide cover, absorbing shots from both sides block by block. **A bonus Blue Whale** periodically crosses the top of the screen for extra points.

The game tracks score, lives, and wave number in a real-time HUD, saves the top 10 scores to an SQLite database between sessions, and progressively increases creature speed and fire rate with each new wave.

4.Core Features

Feature	Description
Submarine Player	Arrow key movement, spacebar torpedo fire with 0.35s cooldown
Enemy Formation	5 rows x 11 columns of sea creatures, synchronized movement, descend on wall contact
Enemy Counter-Attack	Creatures randomly fire torpedoes downward at increasing frequency per wave
Coral Reef Bunkers	4 destructible obstacles built from grids of 3x3 pixel CoralBlocks
Blue Whale Bonus	Spawns from a random side every 10-20 seconds, worth 150 points
Power-Up System	25% drop chance on creature death: Double Torpedo, Shield, Extra Life
Progressive Difficulty	Creature speed +15% per wave (capped at 2.5x), fire interval -0.03s per wave (min 0.12s)
Wave System	2-second wave-clear banner between waves, gameplay freezes during transition
SQLite Leaderboard	Top 10 scores persisted in seaofchaos.db with player name, score, and wave reached
Animated GUI	Scrolling ocean background, sprite-based entities, full HUD, menus, overlays
Sound Effects	Torpedo fire, explosion, ambient ocean music via Raylib audio
Main Menu	Animated start screen with controls and power-up legend

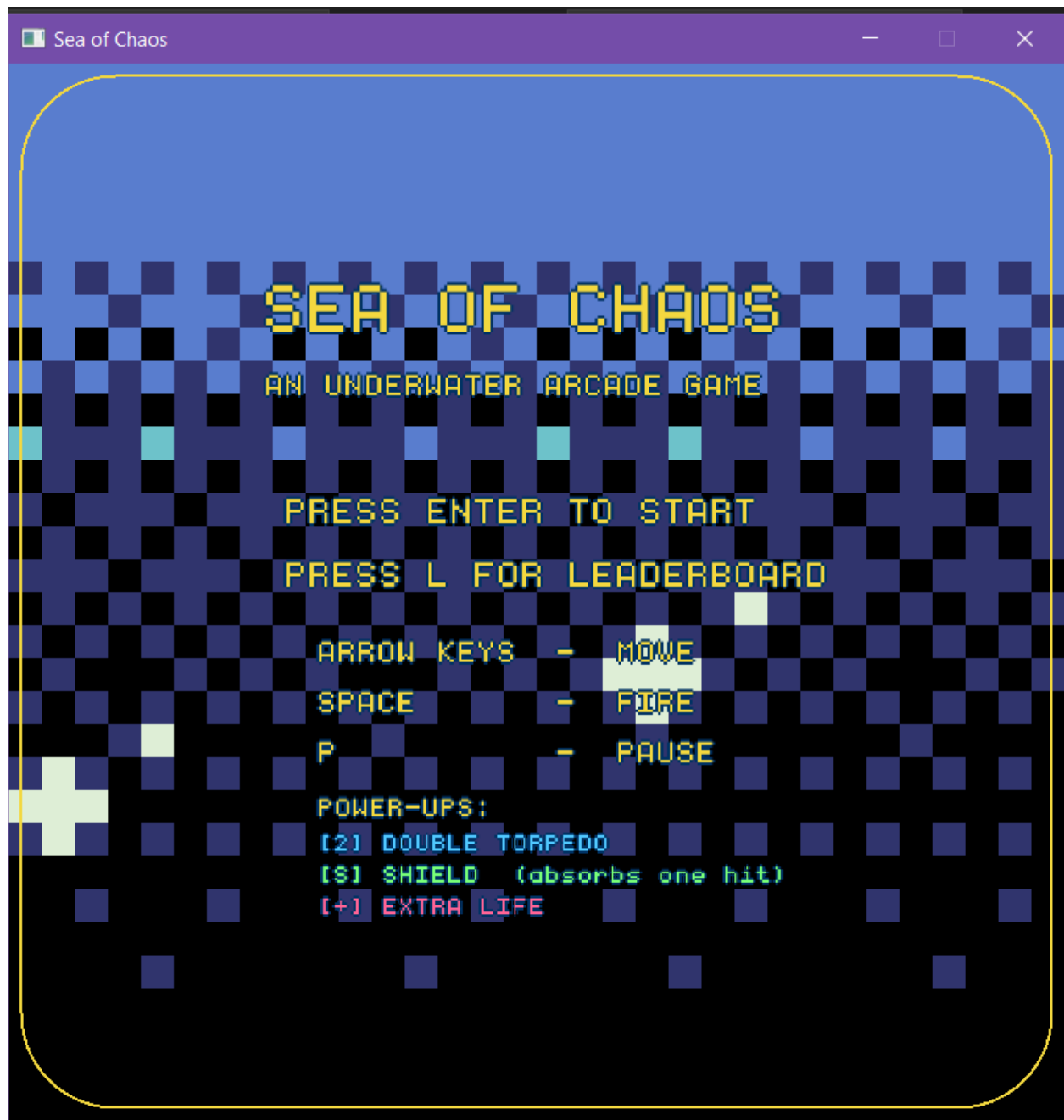


Figure 1: Sea of Chaos main menu showing controls guide and power-up legend

5. Object-Oriented Design

5.1 Class Hierarchy

The project is structured around a clean abstract base class (Entity) from which all game objects derive, alongside three standalone management classes.

Entity (abstract base)



Game (central controller, owns all entities)

ScoreManager (SQLite leaderboard: insert, query, top 10)

UIManager (all rendering: HUD, menus, overlays)

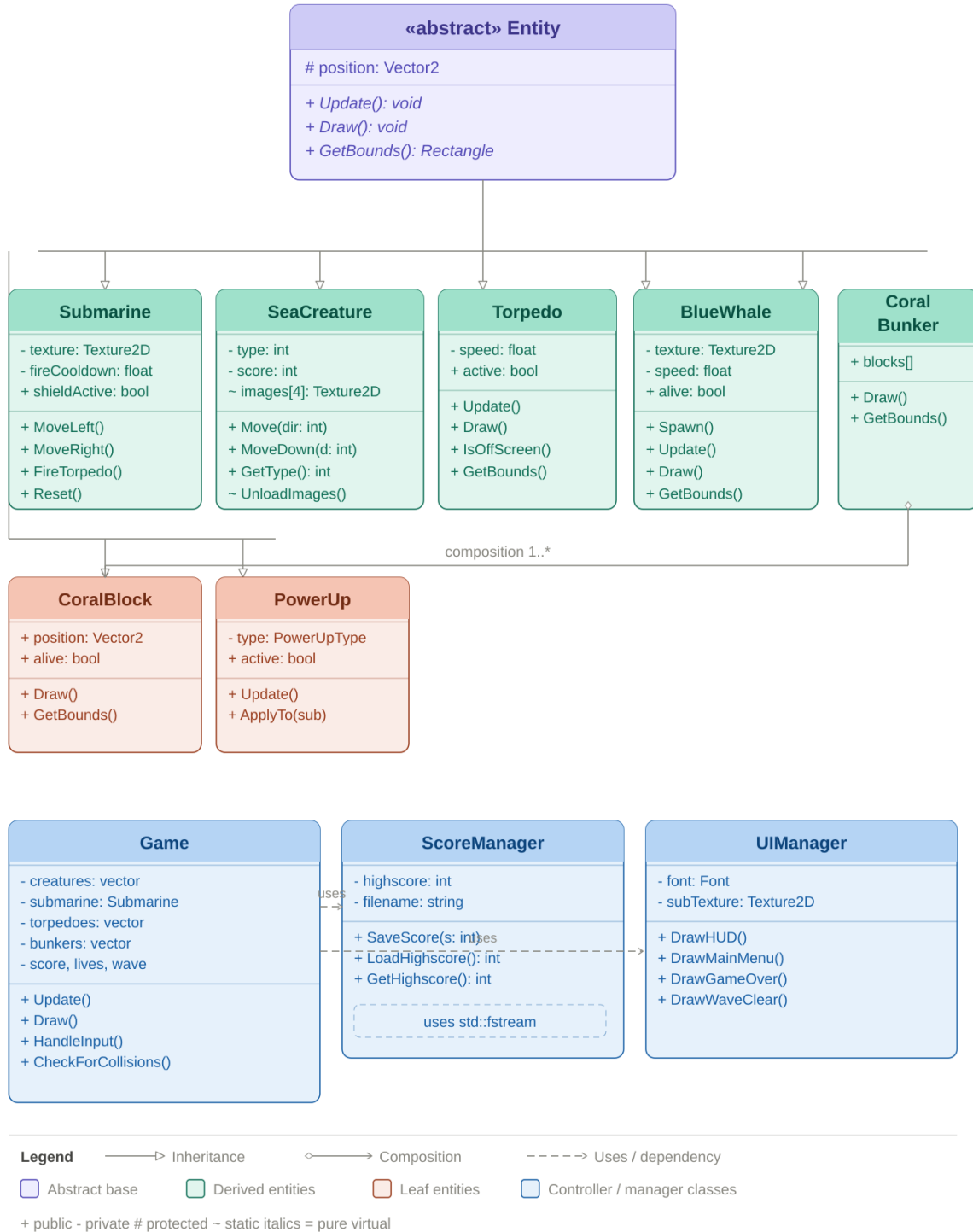


Figure 2: UML Class Diagram, Entity inheritance hierarchy, controller classes (Game, ScoreManager, UIManager), and their relationships.

Class	Responsibility	Key Methods
Entity	Abstract base for all game objects. Holds shared position.	Update(), Draw(), GetBounds()
Submarine	Player entity. Movement, torpedo firing, power-up state.	MoveLeft(), MoveRight(), FireTorpedo(), Reset()
SeaCreature	Enemy entity. Type-specific sprite, formation movement.	Move(), MoveDown(), GetType(), GetPosition()
Torpedo	Projectile for player and enemy. Direction set by speed sign.	Update(), Draw(), IsOffScreen()
BlueWhale	Bonus creature. Spawns randomly, crosses screen.	Spawn(), Update(), Draw()
CoralBlock	Single 3x3 pixel block unit of a bunker.	Draw(), GetBounds()
CoralBunker	Destructible bunker built from a CoralBlock grid.	Draw(), GetBounds()
PowerUp	Collectible item dropped by enemies on death.	Update(), Draw(), ApplyTo()
Game	Central controller. Owns all entities, runs game loop, handles collisions.	Update(), Draw(), HandleInput(), CheckForCollisions()
ScoreManager	All SQLite leaderboard operations.	SaveScore(), GetTopScores()
UIManager	Renders all screens: HUD, main menu, game over, wave-clear.	DrawHUD(), DrawMainMenu(), DrawGameOver(), DrawWaveClear()

5.2 OOP Principles Applied

Abstraction

The **Entity** class is a **pure abstract base class** that defines the interface every game object must implement, without specifying how. It declares **three pure virtual methods**, `Update()`, `Draw()`, and `GetBounds()`, and provides a shared `Vector2` position member. No **Entity** object can be instantiated directly. **UIManager** abstracts all rendering complexity away from the **Game** class. **ScoreManager** abstracts all SQLite database operations, the **Game** class simply calls `SaveScore()` with no knowledge of how or where data is stored.

Encapsulation

Each class manages its own internal state privately. **Submarine's** texture, fire cooldown timer, and sound are private members accessible only through public methods such as `FireTorpedo()`, `MoveLeft()`, and `Reset()`. **ScoreManager** encapsulates its SQLite3 database connection and file path privately, exposing only `SaveScore()` and `GetTopScores()`. **CoralBunker** owns its vector of **CoralBlocks** and manages their construction internally.

Inheritance

All game entities inherit from **Entity**, acquiring a shared position member and committing to implementing the three virtual methods. **Submarine**, **SeaCreature**, **Torpedo**, **BlueWhale**, **CoralBunker**, **CoralBlock**, and **PowerUp** all extend **Entity**. This eliminates code duplication for shared properties and ensures every object integrates cleanly into the game loop through a uniform interface.

Polymorphism

Virtual function dispatch is used throughout the game loop. `Draw()` is called uniformly on all entity types, **SeaCreature::Draw()** selects the correct sprite based on type, **BlueWhale::Draw()** only renders when alive, **PowerUp::Draw()** renders the correct coloured box with label. **Torpedo::Update()** moves in the correct direction based on whether speed is positive or negative, without any special-case logic at the call site.

Constructors and Destructors

Every class with Raylib resources implements a constructor that loads those resources and a destructor that unloads them. Submarine loads its texture and sound in the constructor and unloads both in `~Submarine()`. SeaCreature uses a static shared texture array to avoid redundant loads, textures are loaded once per type and unloaded via the static **UnloadImages() method**. ScoreManager opens the SQLite database connection in its constructor and closes it in the destructor via `sqlite3_close()`. Entity declares a virtual `~Entity() = default` to ensure correct destruction through base pointers.

6. System Architecture

6.1 Game Flow

- **main()** initialises Raylib window, audio device, font, and textures
- **UIManager** renders the animated main menu loop
- Player presses ENTER → **main menu loop** exits
- **Game object constructed** → **InitGame()** sets all state, creates bunkers and creatures
- **Main game loop begins:** **UpdateMusicStream()**, **HandleInput()**, **Update()**, **Draw()**, overlay rendering
- **Torpedo-creature collision** → creature erased, score updated, 25% PowerUp chance, **ScoreManager** saves score if needed
- **Torpedo-submarine collision** → shield absorbs, or lives-- → **GameOver()** if lives == 0
- **Torpedo-bunker collision** → individual **CoralBlock** erased from **bunker.blocks**
- Creatures reach floor or touch submarine → **GameOver()**
- **creatures.empty()** → **waveClearPending** = true, 2-second timer begins
- Timer expires → **LoadNextWave()** increments wave, speed, fire rate
- On **GameOver**, player enters name → **ScoreManager** saves final score with wave reached
- Player presses ENTER on game over → **Reset()** + **InitGame()** restart the game
- **WindowShouldClose()** → resources unloaded, **sqlite3_close()**, **CloseWindow()**



Figure 3: In-game HUD during Wave 2, showing score, lives icons, wave indicator, and active Double Torpedo power-up



Figure 4: Wave-clear screen displayed for 2 seconds between waves

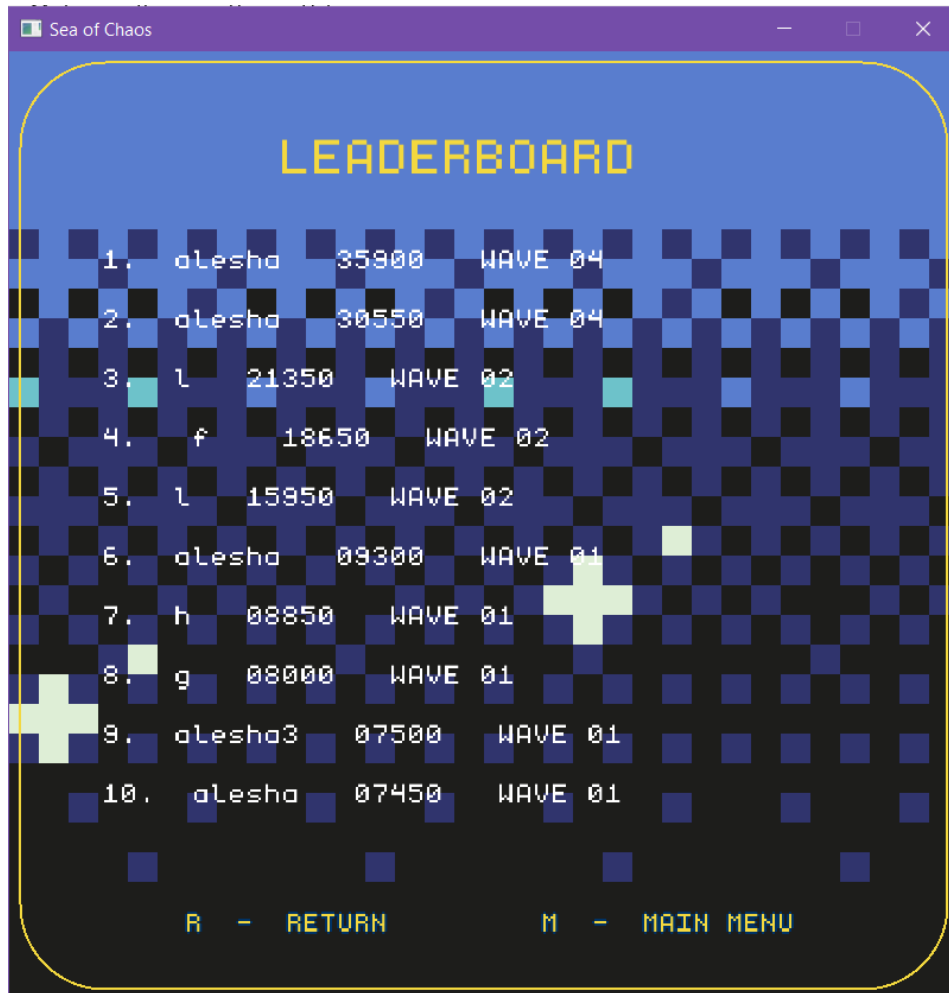


Figure 5: Game over overlay showing final score, top 10 leaderboard, and wave reached

6.2 File Structure

```
SeaOfChaos/  
├── main.cpp  
├── game.cpp / game.hpp  
├── entity.hpp  
├── submarine.cpp / .hpp  
├── seacreature.cpp / .hpp  
├── torpedo.cpp / .hpp  
├── bluewhale.cpp / .hpp  
├── coralblock.cpp / .hpp  
└── coralbunker.cpp / .hpp
```

```
|— powerup.cpp / .hpp
|— scoremanager.cpp / .hpp  <- SQLite leaderboard
|— uimanager.cpp / .hpp
|— Graphics/
|— Sounds/
|— Font/
|— seaofchaos.db           <- SQLite database, auto-created on first run
```

7. Implementation Details

7.1 Entity Base Class

Entity is a pure abstract class and the root of the entire inheritance hierarchy. It declares *three pure virtual methods* (Update, Draw, GetBounds), provides a protected Vector2 position member initialised to {0, 0} in the default constructor, and declares a virtual destructor to ensure correct polymorphic destruction. No game object exists outside this hierarchy.

7.2 Submarine

The player entity. It loads a texture and a fire sound in its constructor and unloads both in its destructor. **MoveLeft()** and **MoveRight()** clamp the submarine to a 25-pixel margin from the screen edges. **FireTorpedo()** enforces a 0.35-second cooldown using **GetTime()**, fires a single centred torpedo, and, if `doubleTorpedoTimer > 0`, fires a second torpedo offset 12 pixels to the right. **Draw()** renders a green shield ring when `shieldActive` is true. **Reset()** re-centres the submarine, clears all torpedoes, and resets power-up state.

7.3 SeaCreature

The enemy entity. *Supports four types: Pufferfish (type 1, 100 pts), Clownfish (type 2, 200 pts), Shark (type 3, 300 pts), and a fourth fish (type 4, 150 pts).* Textures are stored in a static array `creatureImages[4]` shared across all instances, loaded once per type. `Move(int direction)` shifts `position.x`, and `MoveDown(int distance)` shifts `position.y`. Formation movement is driven externally by `Game::MoveCreatures()`. The static `UnloadImages()` method unloads all four textures on game shutdown.

7.4 Torpedo

Used for both player and enemy projectiles. The sign of speed determines direction: **negative moves upward (player), positive moves downward (enemy).** `Update()` adds speed to `position.y` each frame and deactivates the torpedo if `IsOffScreen()` returns true, which checks both the top (`< 25`) and bottom (`> screenHeight - 100`) boundaries. `Draw()` renders a 4x15 cyan rectangle only while active is true.

7.5 BlueWhale

The bonus creature. `Spawn()` randomly selects a side: left side sets `position.x = 25` and `speed = 3`; right side sets `position.x` to the far edge and `speed = -3`. `Update()` moves by speed each frame and deactivates when it exits the opposite boundary. `GetBounds()` returns a zero-size rectangle when alive is false, preventing false collision detections. Worth 150 points on hit.

7.6 CoralBunker & CoralBlock

CoralBlock is a minimal entity representing a single 3x3 pixel block rendered in coral-pink. **CoralBunker** builds its shape from a static 13-row x 23-column binary grid, constructing one **CoralBlock** per '1' cell offset from the bunker's origin position. The grid creates an arch shape with a hollow bottom centre, matching the classic Space Invaders bunker silhouette. Individual blocks are erased from `bunker.blocks` directly in `Game::CheckForCollisions()` upon torpedo or creature contact.

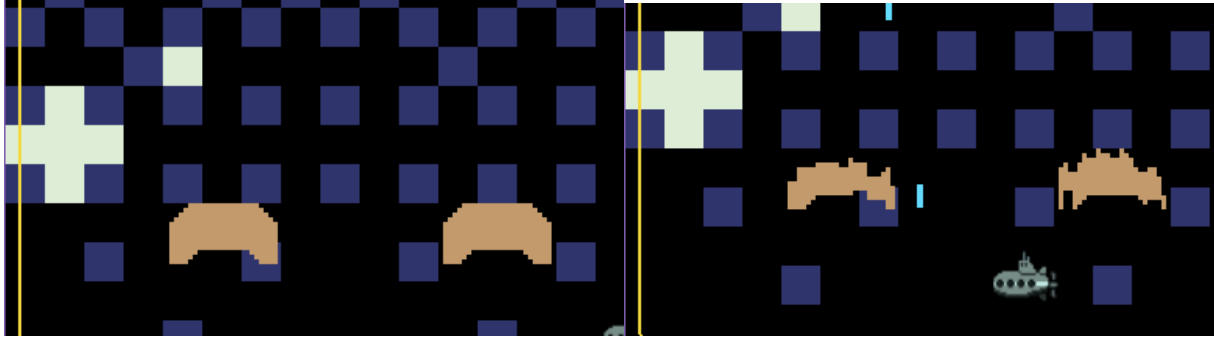


Figure 7: CoralBunker showing intact state (left) and block-level damage after absorbing torpedo fire (right)

7.7 PowerUp

Three types: DOUBLE_TORPEDO (cyan, sets doubleTorpedoTimer = 8.0 seconds), SHIELD (green, sets shieldActive = true), EXTRA_LIFE (pink, increments lives).

Power-ups fall downward at FALL_SPEED = 2.0f pixels per frame and deactivate if they reach the bottom without being collected.

Draw() renders a 16x16 coloured box with a single character label ("2", "S", "+").

ApplyTo(Submarine& sub, int& lives) applies the effect and sets active = false. The Game spawns a power-up with a 25% chance when a creature is destroyed.

7.8 ScoreManager

ScoreManager handles persistent data using both a plain text file (highscore.txt) for the single highest score and an SQLite3 database (seaofchaos.db) for the full leaderboard. On construction, it calls ReadFromFile() to load the cached high score from highscore.txt and InitDB() to open or create the SQLite database

InitDB() executes sqlite3_open("seaofchaos.db", &db) followed by a CREATE TABLE IF NOT EXISTS statement defining columns: id (primary key), name (TEXT), score (INTEGER), wave (INTEGER), and timestamp (TEXT with default CURRENT_TIMESTAMP). The timestamp is automatically recorded by SQLite when each entry is inserted.

SaveScore(int score) compares the given score to the cached high score and writes to highscore.txt only if a new high score is achieved, using std::ofstream.

SaveToDB(const std::string& playerName, int score, int wave) executes a parameterised INSERT INTO statement using sqlite3_prepare_v2() and sqlite3_bind_*() to safely bind the three values. This is called from Game::GameOver() after the player enters their name.

GetTopScores(int limit) executes a SELECT name, score, wave, timestamp FROM leaderboard ORDER BY score DESC LIMIT ? query and returns the results as a vector of LeaderboardEntry structs. The limit parameter (default 10) is bound using sqlite3_bind_int().

The destructor calls sqlite3_close() to release the database connection cleanly. All SQLite error codes are checked and reported to std::cerr without crashing the game.

7.9 UIManager

Handles all **screen rendering**, keeping Game free of any direct UI drawing logic. Constructed with a Font and a submarine **Texture2D** for the lives display.

DrawHUD() renders: an outer border, score at the top, lives as submarine icons at the bottom-left, wave number at the bottom-right, and power-up indicators in the bottom centre when active. **DrawMainMenu()** renders the title, subtitle, controls guide, and power-up legend. **DrawGameOver()** overlays a semi-transparent black panel with final score, wave reached, and the top 10 leaderboard. **DrawWaveClear()** overlays a banner with the completed wave number and the next wave number.

7.10 Game

The central controller. **InitGame()** resets all state variables, creates four bunkers, and creates the 5x11 creature grid. **Update()** ticks through: wave-clear pause timer, whale spawn interval, submarine update, torpedo updates, power-up updates, creature movement, enemy fire, cleanup of inactive torpedoes and power-ups, Blue Whale update, and full collision detection. **LoadNextWave()** increments the wave

counter, increases creatureSpeedMultiplier by 0.15 (capped at 2.5), decreases enemyFireInterval by 0.03 (capped at 0.12), and repopulates creatures. **Collision detection** handles six interaction pairs: player torpedoes vs. creatures, player torpedoes vs. bunker blocks, player torpedoes vs. Blue Whale, enemy torpedoes vs. submarine (with shield check), enemy torpedoes vs. bunker blocks, creatures vs. bunker blocks, creatures vs. submarine, and power-ups vs. submarine.

8. Code Snippets

This section presents five representative excerpts that demonstrate the OOP principles at work in the implementation.

Snippet 1, Abstract Entity Base Class

```
#pragma once
#include <raylib.h>

class Entity { // Base class for all game entities
public:
    virtual void Update() = 0; //virtual functions to be implemented by derived
    classes
    virtual void Draw() = 0;
    virtual Rectangle GetBounds() = 0;
    virtual ~Entity() = default;
    Entity() : position({ 0, 0 }) {}
protected:
    Vector2 position; // Common position property for all entities
```

Snippet 2, Runtime Polymorphism in the Game Loop

```
void Game::Draw()
{
    submarine.Draw();

    for (auto& torpedo : submarine.torpedoes) {
        torpedo.Draw();
    }

    for (auto& bunker : bunkers) {
        bunker.Draw();
    }

    for (auto& creature : creatures) {
```

```

        creature.Draw();
    }

    for (auto& torpedo : enemyTorpedoes) {
        torpedo.Draw();
    }

    for (auto& pu : powerUps) {
        pu.Draw();
    }

    blueWhale.Draw();
}

```

Snippet 3, Encapsulation and Constructors/Destructors

```

//submarine.hpp
#pragma once
#include <raylib.h>
#include <vector>
#include "torpedo.hpp"
#include "entity.hpp"

class Submarine : public Entity { // Represents the submarine, can move left/right
    and fire torpedoes
public:
    Submarine();
    ~Submarine();
    void Update() override;
    void Draw() override;
    Rectangle GetBounds() override;
    void MoveLeft();
    void MoveRight();
    void FireTorpedo();
    void Reset();
    std::vector<Torpedo> torpedoes;

    // Power-up state - written directly by PowerUp::ApplyTo()
    double doubleTorpedoTimer; // >0 means double-fire is active
    bool shieldActive; // true = next hit is absorbed

private:
    Texture2D image; // Submarine image texture
    double lastFireTime;
    Sound torpedoSound;
};

//submarine.cpp
#include "submarine.hpp"

Submarine::Submarine() // Constructor loads the submarine image and initializes
state
{
    image = LoadTexture("Graphics/submarine.png");
    position.x = (GetScreenWidth() - image.width) / 2;
}

```

```

    position.y = GetScreenHeight() - image.height - 100;
    lastFireTime = 0.0;
    torpedoSound = LoadSound("Sounds/laser.ogg");
    doubleTorpedoTimer = 0.0;
    shieldActive = false;
}

Submarine::~Submarine() // Destructor unloads the texture and sound resources
{
    UnloadTexture(image);
    UnloadSound(torpedoSound);}

```

Snippet 4, SQLite Leaderboard with ScoreManager

***Note:** Replace with your actual code from: scoremanager.hpp and scoremanager.cpp*

```

// scoremanager.hpp
...
// SQLite methods
void SaveToDB(const std::string& playerName, int score, int wave);
std::vector<LeaderboardEntry> GetTopScores(int limit = 10);

private:
    std::string filename;
    int highscore;
    sqlite3* db; // SQLite database connection

    // Helper methods for file I/O and database initialization
    void WriteToFile(int value);
    int ReadFromFile();
    void InitDB();
};

// scoremanager.cpp
#include "scoremanager.hpp"
#include <fstream>
#include <iostream>

// Constructor initializes the score manager
ScoreManager::ScoreManager(const std::string& filename)
{
    this->filename = filename;
    highscore = ReadFromFile();
    db = nullptr;
    InitDB();
}

ScoreManager::~ScoreManager()
{
    if (db) sqlite3_close(db); // Clean up the database connection when the
ScoreManager is destroyed
}

```

```

// Initializes the SQLite database and creates the leaderboard table if it doesn't
exist
void ScoreManager::InitDB()
{
    int rc = sqlite3_open("seaofchaos.db", &db);
    if (rc != SQLITE_OK) {
        std::cerr << "Cannot open database: " << sqlite3_errmsg(db) << std::endl;
        db = nullptr;
        return;
    }

    // Create the leaderboard table if it doesn't exist
    const char* sql =
        "CREATE TABLE IF NOT EXISTS leaderboard ("
        "id INTEGER PRIMARY KEY AUTOINCREMENT,"
        "name TEXT NOT NULL,"
        "score INTEGER NOT NULL,"
        "wave INTEGER NOT NULL,"
        "timestamp TEXT DEFAULT CURRENT_TIMESTAMP"
        ");";

    // Execute the SQL statement to create the table, and check for errors
    char* errMsg = nullptr;
    sqlite3_exec(db, sql, nullptr, nullptr, &errMsg);
    if (errMsg) {
        std::cerr << "SQL error: " << errMsg << std::endl;
        sqlite3_free(errMsg);
    }
}

// Saves a player's score to the SQLite database with their name, score, wave, and
timestamp
void ScoreManager::SaveToDB(const std::string& playerName, int score, int wave)
{
    if (!db) {
        std::cerr << "Database not initialized!" << std::endl;
        return;
    }

    const char* sql = "INSERT INTO leaderboard (name, score, wave) VALUES (?, ?,
?);";
    sqlite3_stmt* stmt; // Prepared statement for inserting a new score

    int rc = sqlite3_prepare_v2(db, sql, -1, &stmt, nullptr);
    if (rc != SQLITE_OK) {
        std::cerr << "Failed to prepare statement: " << sqlite3_errmsg(db) <<
std::endl;
        return;
    }

    // Bind the player name, score, and wave to the prepared statement parameters
    sqlite3_bind_text(stmt, 1, playerName.c_str(), -1, SQLITE_STATIC);
    sqlite3_bind_int(stmt, 2, score);
    sqlite3_bind_int(stmt, 3, wave);

```

```

        rc = sqlite3_step(stmt);
        if (rc != SQLITE_DONE) {
            std::cerr << "Failed to insert data: " << sqlite3_errmsg(db) << std::endl;
        }

        sqlite3_finalize(stmt); // Clean up the prepared statement after execution
    }

    // Retrieves the top scores from the SQLite database
    std::vector<LeaderboardEntry> ScoreManager::GetTopScores(int limit)
    {
        std::vector<LeaderboardEntry> entries;
        if (!db) return entries;

        const char* sql = "SELECT name, score, wave, timestamp FROM leaderboard ORDER BY
score DESC LIMIT ?;";
        sqlite3_stmt* stmt;

        int rc = sqlite3_prepare_v2(db, sql, -1, &stmt, nullptr);
        if (rc != SQLITE_OK) {
            std::cerr << "Failed to prepare select statement: " << sqlite3_errmsg(db) <<
std::endl;
            return entries;
        }

        sqlite3_bind_int(stmt, 1, limit);

        // Execute the query and populate the entries vector with the results
        while (sqlite3_step(stmt) == SQLITE_ROW) {
            LeaderboardEntry entry;
            entry.playerName = reinterpret_cast<const char*>(sqlite3_column_text(stmt,
0));
            entry.score = sqlite3_column_int(stmt, 1);
            entry.wave = sqlite3_column_int(stmt, 2);
            entry.timestamp = reinterpret_cast<const char*>(sqlite3_column_text(stmt,
3));
            entries.push_back(entry);
        }

        sqlite3_finalize(stmt);
        return entries;
    }

    // Saves the score to the file if it's a new high score, and updates the in-memory
    high score
    void ScoreManager::SaveScore(int score)
    {
        if (score > highscore) {
            highscore = score;
            WriteToFile(highscore);
        }
    }

    int ScoreManager::LoadHighscore()
    {
        highscore = ReadFromFile();
    }

```



```

        return highscore;
    }

    // Returns the current high score stored in memory
    int ScoreManager::GetHighscore() const
    {
        return highscore;
    }

    // Writes the high score to the file
    void ScoreManager::WriteToFile(int value)
    {
        std::ofstream file(filename);
        if (file.is_open()) {
            file << value;
            file.close();
        }
    }

    // Reads the high score from the file
    int ScoreManager::ReadFromFile()
    {
        int loaded = 0;
        std::ifstream file(filename);
        if (file.is_open()) {
            file >> loaded;
            file.close();
        }
        return loaded;
    }

```

9. Testing

The following test cases were executed manually to verify correct game behaviour across all major systems. All tests were conducted on the final build.

#	Test Case	Input / Condition	Expected Result	Actual Result	Status
T-01	Submarine boundary, left edge	Hold LEFT arrow to screen edge	Submarine stops at $x = 25$	As expected	PASS
T-02	Submarine boundary, right edge	Hold RIGHT arrow to screen edge	Submarine stops at right margin	As expected	PASS
T-03	Torpedo fire cooldown	Press SPACE rapidly	One torpedo fires per 0.35 seconds	As expected	PASS
T-04	Double torpedo power-up	Collect DOUBLE_TORPEDO, fire	Two torpedoes fire for 8 seconds	As expected	PASS
T-05	Shield absorption	Collect SHIELD; take enemy hit	Enemy torpedo absorbed, no life lost	As expected	PASS
T-06	Extra life power-up	Collect EXTRA_LIFE	lives counter increments by 1	As expected	PASS
T-07	Enemy torpedo kills submarine	Allow unshielded enemy hit	lives--; GameOver if $\text{lives} == 0$	As expected	PASS
T-08	Creature reaches floor	Allow creatures to advance to floor	GameOver triggered immediately	As expected	PASS
T-09	Bunker block-level damage	Fire torpedo at bunker	Individual CoralBlock erased on hit	As expected	PASS

10. Data Storage

Persistent data storage uses two complementary systems: a plain text file for the single highest score (**highscore.txt**), and an **SQLite3** database for the full leaderboard (**seaofchaos.db**).

On startup, **ScoreManager** opens **highscore.txt** with `std::ifstream` and reads the stored integer into its `highscore` member. It also opens (or creates) **seaofchaos.db** using `sqlite3_open()` and executes a `CREATE TABLE IF NOT EXISTS` statement. The leaderboard table includes an auto-incrementing `id`, `name` (TEXT), `score` (INTEGER), `wave` (INTEGER), and `timestamp` (TEXT with `DEFAULT CURRENT_TIMESTAMP`).

When a game session ends, the player enters their name and `Game::GameOver()` calls `SaveToDB()` which executes a parameterised `INSERT INTO` statement using `sqlite3_prepare_v2()` and `sqlite3_bind_*`. Parameterised queries prevent SQL injection even with player-supplied names.

The leaderboard is queried using `SELECT name, score, wave, timestamp FROM leaderboard ORDER BY score DESC LIMIT ?`, where the limit is bound as a parameter (default 10). This returns results pre-sorted from highest to lowest score. `UIManager` receives this data as a vector of `LeaderboardEntry` structs and renders it on both the game over screen and the main menu leaderboard view.

Additionally, `SaveScore()` maintains a separate **highscore.txt** file containing only the single highest score ever achieved. This provides quick access to the "best" score without querying the database.

SQLite was chosen because it supports structured queries, scales cleanly with additional columns, and automatically timestamps entries. The database connection is closed in `ScoreManager`'s destructor via `sqlite3_close()`.

11. Technology Stack

Component	Technology	Purpose
Language	C++17	All OOP class design, game logic
Game / GUI	Raylib 5.0	Window, 2D sprite rendering, input, audio
Build System	Makefile	Compilation and linking
Data Storage	SQLite3	Persistent leaderboard database storing top 10 scores with player name, score, and wave reached
IDE	VS Code + MinGW/GCC	Development and debugging
Version Control	Git + GitHub	Source control and progress tracking

Github repository: <https://github.com/seaics123-arch/00PFinalProj>

12. Challenges and Solutions

Circular Include Between PowerUp and Submarine

`PowerUp::ApplyTo()` needs to write to `Submarine` members, but including `submarine.hpp` inside `powerup.hpp` would create a circular dependency. This was resolved using a forward declaration (`class Submarine;`) in `powerup.hpp` and including `submarine.hpp` only in `powerup.cpp`, where the full type is needed. This correctly separates declaration-time and definition-time dependencies.

Static Texture Sharing in SeaCreature

Loading a texture for every `SeaCreature` instance, 55 creatures per wave, would be wasteful and cause resource issues. The solution was a static `Texture2D creatureImages[4]` array shared across all instances. Each texture is loaded only once, checked via `creatureImages[type - 1].id == 0`, and unloaded once at shutdown via the static `UnloadImages()` method. This illustrates a practical application of static class members.

Wave-Clear Pause Without Blocking the Game Loop

Freezing gameplay for the wave-clear banner cannot use `sleep()` or any blocking call, as that would freeze Raylib's window event processing. The solution was a `waveClearPending` boolean flag and a `waveClearTimer` countdown decremented by `GetFrameTime()` each frame. While pending, `Update()` and `HandleInput()` return early, but the main loop continues drawing the banner overlay on each frame. This is a standard game-loop pattern for non-blocking state transitions.

Formation Movement Edge Detection

The creatures move as individual objects rather than a single grid entity, so edge detection cannot be done on a single bounding box. Each frame, `MoveCreatures()`

checks every creature's x position against the screen margins. If any creature's right edge exceeds the right boundary, creaturesDirection flips to -1 and all creatures move down. This correctly reproduces the classic Space Invaders edge-detect-then-descend behaviour using per-object position checks.

Shield Visual Feedback

The shield needed a visible indicator on the submarine without adding a new entity or a complex state machine. The solution was a simple conditional in Submarine::Draw(): if shieldActive is true, DrawCircleLines() renders a green ring centred on the submarine sprite before the sprite itself is drawn. This demonstrates how draw-time state inspection can replace the overhead of a separate visual entity.

SQLite Integration and Parameterised Queries

Integrating SQLite3 required adding the library to the build system and learning the C API for prepared statements. The challenge was ensuring safe insertion of player-supplied names without risking SQL injection. The solution uses sqlite3_prepare_v2() and sqlite3_bind_text() to bind the name string as a parameter rather than concatenating it into the SQL string. This is the standard secure way to handle user input with SQLite.

Deciding Between a Monolithic Class and Modular Separation

An early design question was whether to put all game logic inside a single Game class, as is common in beginner projects. The decision to separate rendering into UIManager and database operations into ScoreManager was deliberate. This separation means that changes to the HUD layout never risk breaking game logic, and the storage strategy can be swapped without touching the game loop. This reflects the Single Responsibility Principle and mirrors professional software design practice.

13. Scope and Limitations

In Scope

- Full Space Invaders gameplay loop with underwater theme and four enemy creature types
- Player submarine with torpedo firing, lives system, double-torpedo and shield power-ups, extra life
- Five-row, eleven-column enemy formation with synchronized movement and progressive difficulty
- Four destructible CoralBunker obstacles with individual block-level damage
- BlueWhale bonus creature with randomized spawn intervals
- Multiple waves with increasing creature speed and enemy fire rate
- Animated Raylib GUI: main menu, in-game HUD, wave-clear banner, game over overlay
- Persistent SQLite leaderboard storing the top 10 scores across sessions, each entry recording player name, score, and wave reached
- Sound effects for torpedo fire and explosions, ambient background music

Out of Scope (Current Version)

- Multiplayer or network functionality
- 3D graphics or physics engine
- Mobile or web platform support
- Procedurally generated or editable level layouts
- Player name editing after entry

Known Limitations

- The leaderboard stores only the top 10 scores; no per-session score history beyond that is retained

- Enemy AI is purely probabilistic; creatures do not aim at the player's current position
- Formation edge detection performs a full linear scan of all creatures every frame; this is acceptable at 55 creatures but would not scale to hundreds

14. Conclusion

Sea of Chaos successfully demonstrates all required Object-Oriented Programming principles through a complete, fully playable 2D arcade game. **The abstract Entity** base class provides the foundation for a clean inheritance hierarchy from which seven distinct entity types derive. **Encapsulation** is consistently enforced across all classes through private members and controlled public interfaces. Runtime **polymorphism** through C++ virtual dispatch drives the game loop uniformly across all entity types without special-case branching. Persistent data storage through **SQLite3** provides a structured, queryable leaderboard that retains the top 10 scores across sessions, going beyond flat file storage to demonstrate real-world database integration in a C++ application.

Sea of Chaos is a genuinely playable game with multiple waves of increasing difficulty, a power-up system with three distinct effects, destructible obstacles, a bonus enemy, and a complete user interface, all built from **scratch in C++ using Raylib 5.0**. The modular class design means each component can be developed, tested, and extended independently, which reflects professional software engineering practice.

This project demonstrates that **OOP is not a theoretical abstraction** but the natural architecture for any system composed of many distinct objects that share a common interface. When the problem is right, the pattern emerges on its own.

15. References

- Raylib Development Team. (2023). Raylib 5.0, A simple and easy-to-use library to enjoy videogames programming. <https://www.raylib.com>
- Stroustrup, B. (2013). The C++ programming language (4th ed.). Addison-Wesley.
- Lafore, R. (2002). Object-oriented programming in C++ (4th ed.). Sams Publishing. (Course reference text)
- Taito Corporation. (1978). Space Invaders [Arcade game]. Taito Corporation. (Original game this project is inspired by)
- SQLite Consortium. (2024). SQLite, C library reference. <https://www.sqlite.org/docs.html>
- cppreference.com. (2024). Virtual functions, C++ reference. <https://en.cppreference.com/w/cpp/language/virtual>